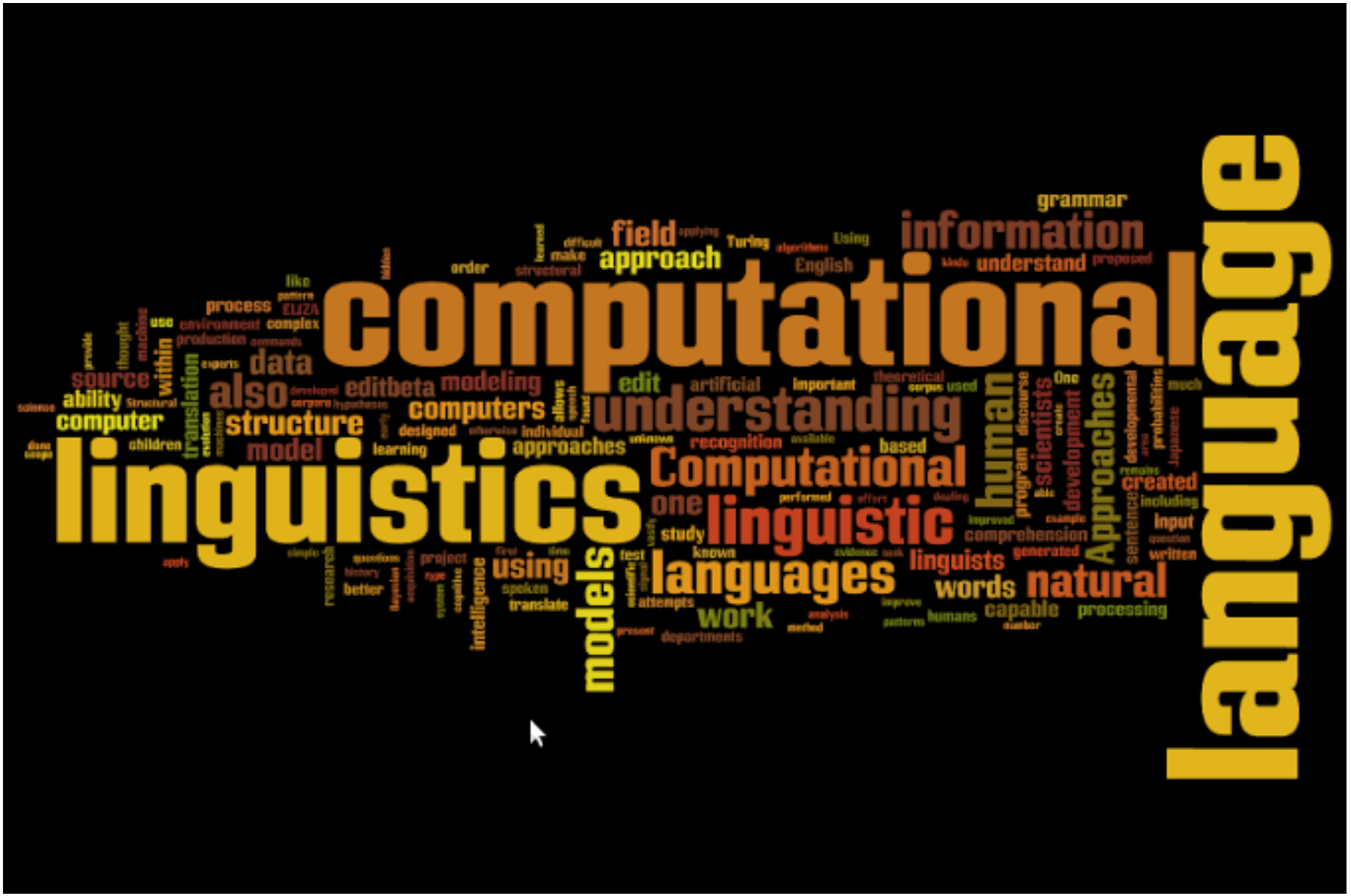
INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

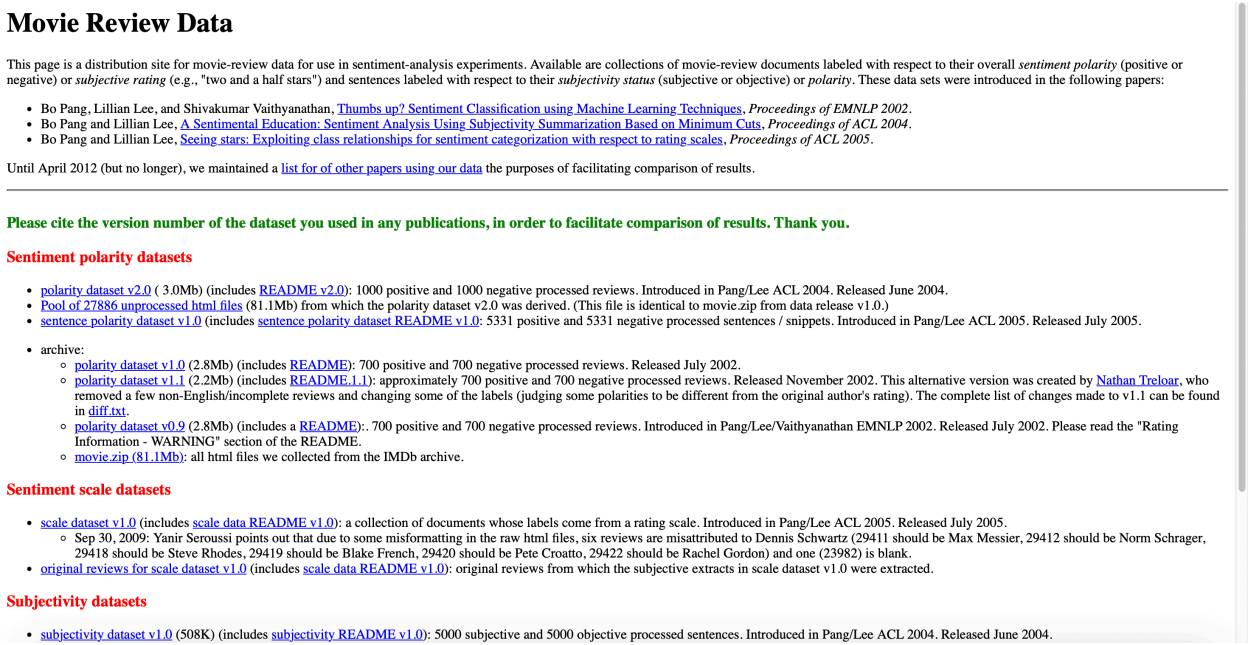
Name	Bhargav Shukla
Email id	bhargav.shukla@research.iiit.ac.in

2025114003

Computational Linguistics-2

Assignment-3





Movie Review Data

This page is a distribution site for movie-review data for use in sentiment-analysis experiments. Available are collections of movie-review documents labeled with respect to their overall *sentiment polarity* (positive or negative) or *subjective rating* (e.g., "two and a half stars") and sentences labeled with respect to their *subjectivity status* (subjective or objective) or *polarity*. These data sets were introduced in the following papers:

- Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan, Thumbs up? Sentiment Classification using Machine Learning Techniques, *Proceedings of EMNLP 2002*.
- Bo Pang and Lillian Lee, A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts, *Proceedings of ACL 2004*.
- Bo Pang and Lillian Lee, Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales, *Proceedings of ACL 2005*.

Until April 2012 (but no longer), we maintained a list for of other papers using our data the purposes of facilitating comparison of results.

Please cite the version number of the dataset you used in any publications, in order to facilitate comparison of results. Thank you.

Sentiment polarity datasets

- polarity dataset v2.0 (3.0Mb) (includes README v2.0): 1000 positive and 1000 negative processed reviews. Introduced in Pang/Lee ACL 2004. Released June 2004.
- Pool of 27886 unprocessed html files (81.1Mb) from which the polarity dataset v2.0 was derived. (This file is identical to movie.zip from data release v1.0.)
- sentence polarity dataset v1.0 (includes sentence polarity dataset README v1.0: 5331 positive and 5331 negative processed sentences / snippets. Introduced in Pang/Lee ACL 2005. Released July 2005.

- archive:
  - polarity dataset v1.0 (2.8Mb) (includes README): 700 positive and 700 negative processed reviews. Released July 2002.
  - polarity dataset v1.1 (2.2Mb) (includes README.1.1): approximately 700 positive and 700 negative processed reviews. Released November 2002. This alternative version was created by Nathan Treloar, who removed a few non-English/incomplete reviews and changing some of the labels (judging some polarities to be different from the original author's rating). The complete list of changes made to v1.1 can be found in diff.txt.
  - polarity dataset v0.9 (2.8Mb) (includes a README):. 700 positive and 700 negative processed reviews. Introduced in Pang/Lee/Vaithyanathan EMNLP 2002. Released July 2002. Please read the "Rating Information - WARNING" section of the README.
  - movie.zip (81.1Mb): all html files we collected from the IMDb archive.

Sentiment scale datasets

- scale dataset v1.0 (includes scale data README v1.0): a collection of documents whose labels come from a rating scale. Introduced in Pang/Lee ACL 2005. Released July 2005.
  - Sep 30, 2009: Yanir Seroussi points out that due to some misformatting in the raw html files, six reviews are misattributed to Dennis Schwartz (29411 should be Max Messier, 29412 should be Norm Schrager, 29418 should be Steve Rhodes, 29419 should be Blake French, 29420 should be Pete Croatto, 29422 should be Rachel Gordon) and one (23982) is blank.
- original reviews for scale dataset v1.0 (includes scale data README v1.0): original reviews from which the subjective extracts in scale dataset v1.0 were extracted.

Subjectivity datasets

- subjectivity dataset v1.0 (508K) (includes subjectivity README v1.0): 5000 subjective and 5000 objective processed sentences. Introduced in Pang/Lee ACL 2004. Released June 2004.

https://www.cs.cornell.edu/people/pabo/movie-review-data/

Lots of datasets!! Which one do we use ?

We use the polarity dataset v2.0, as we want to make a classifier that identifies a movie in terms of positive or negative review instead of a scale or something else.

Training and Testing Data Split

We use 10-fold data training and testing split. It works in following way:

- First 200 sentences (100 positive and 100 negative) are used as testing set, and the remaining 1800 sentences (900 positive and 900 negative) are used as the training set.
- After that we use the next set of 200 sentences as testing, and the remaining 1800 sentences as the training set, completely rerunning our program and observing output we get this time.
- Thus, our experiment will continue 10 times, where each time, 200 sentences will be categorized as training sentences and remaining 200 as the test sentence.

Motivation behind using the 10-fold approach

- There is no clear demarkation between training and testing dataset we are using, and hence this way we consider every possible split between the test and training sentences, making our model much more accurate.
- The final results got in each of the 10-folds can be averaged out, or something similar to get our final significant output

Note that the training and testing done for each fold is independent from previous ones, as we train and test from scratch each time.
This ensures that the new testing dataset was not already used to train the model already, and preserve the authenticity of test data set during each run.

After successful execution of the code on the dataset, the below showed spreadsheet the data we got for each of the subparts.

Q1)

i) In this representation, each review is represented using a Bag-of-Words vector. The value of a feature is the number of times the corresponding word occurs in the review. The vocabulary is constructed from the training data only, so words that occur only in the test set are excluded.

```
================================================================================
FEATURE 1: BOW frequency (train vocabulary)
================================================================================

Fold 1/10
  Vocabulary size: 39,862
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[77, 23], [21, 79]]
Precision : 0.7745
Recall    : 0.7900
Accuracy  : 0.7800
F1        : 0.7822

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[89, 11], [31, 69]]
Precision : 0.8625
Recall    : 0.6900
Accuracy  : 0.7900
F1        : 0.7667

Fold 2/10
  Vocabulary size: 40,022
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[85, 15], [20, 80]]
Precision : 0.8421
Recall    : 0.8000
Accuracy  : 0.8250
F1        : 0.8205

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[86, 14], [21, 79]]
Precision : 0.8495
Recall    : 0.7900
Accuracy  : 0.8250
F1        : 0.8187
```

Output of the feature for first two folds

```
--------------------------------------------------------------------------------
10-FOLD AGGREGATE RESULTS: FEATURE 1
--------------------------------------------------------------------------------

Naive Bayes
-----------
Confusion matrix [[TN, FP], [FN, TP]] = [[828, 172], [206, 794]]
Precision : 0.8219
Recall    : 0.7940
Accuracy  : 0.8110
F1        : 0.8077

Logistic Regression
-------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[837, 163], [162, 838]]
Precision : 0.8372
Recall    : 0.8380
Accuracy  : 0.8375
F1        : 0.8376
```

Average score of feature across all ten folds.

ii) The second representation is also a frequency-based Bag-of-Words representation, but the vocabulary is constructed using the union of the training and test vocabularies. Thus, words that occur in the test set but not in the training set are included in the feature space. Importantly, the test labels and test word frequencies are not used to train the classifiers.

```
================================================================================
FEATURE 2: BOW frequency (train + test vocabulary)
================================================================================

Fold 1/10
  Vocabulary size: 41,644
  Train=1800 Test=200

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[77, 23], [23, 77]]
Precision : 0.7700
Recall    : 0.7700
Accuracy  : 0.7700
F1        : 0.7700

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[89, 11], [31, 69]]
Precision : 0.8625
Recall    : 0.6900
Accuracy  : 0.7900
F1        : 0.7667

Fold 2/10
  Vocabulary size: 41,644
  Train=1800 Test=200

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[86, 14], [21, 79]]
Precision : 0.8495
Recall    : 0.7900
Accuracy  : 0.8250
F1        : 0.8187

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[86, 14], [21, 79]]
Precision : 0.8495
Recall    : 0.7900
Accuracy  : 0.8250
F1        : 0.8187
```

```
--------------------------------------------------------------------------------
10-FOLD AGGREGATE RESULTS: FEATURE 2
--------------------------------------------------------------------------------

Naive Bayes
-----------
Confusion matrix [[TN, FP], [FN, TP]] = [[834, 166], [210, 790]]
Precision : 0.8264
Recall    : 0.7900
Accuracy  : 0.8120
F1        : 0.8078

Logistic Regression
-------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[837, 163], [162, 838]]
Precision : 0.8372
Recall    : 0.8380
Accuracy  : 0.8375
F1        : 0.8376
```

iii)The third representation uses a Bag-of-Words model in which each word is represented by a binary value rather than its frequency. Thus, the feature value is 1 if a word occurs at least once in the review and 0 otherwise. Therefore, multiple occurrences of the same word do not increase its feature value. The vocabulary is constructed from the training data, and words that are unseen during training are excluded from the feature space.

```
================================================================================
FEATURE 3: BOW binary
================================================================================

Fold 1/10
  Vocabulary size: 39,862
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[81, 19], [24, 76]]
Precision : 0.8000
Recall    : 0.7600
Accuracy  : 0.7850
F1        : 0.7795

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[87, 13], [16, 84]]
Precision : 0.8660
Recall    : 0.8400
Accuracy  : 0.8550
F1        : 0.8528

Fold 2/10
  Vocabulary size: 40,022
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[86, 14], [23, 77]]
Precision : 0.8462
Recall    : 0.7700
Accuracy  : 0.8150
F1        : 0.8063

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[85, 15], [16, 84]]
Precision : 0.8485
Recall    : 0.8400
Accuracy  : 0.8450
F1        : 0.8442
```

```
--------------------------------------------------------------------------------
10-FOLD AGGREGATE RESULTS: FEATURE 3
--------------------------------------------------------------------------------

Naive Bayes
-----------
Confusion matrix [[TN, FP], [FN, TP]] = [[856, 144], [210, 790]]
Precision : 0.8458
Recall    : 0.7900
Accuracy  : 0.8230
F1        : 0.8170

Logistic Regression
-------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[871, 129], [138, 862]]
Precision : 0.8698
Recall    : 0.8620
Accuracy  : 0.8665
F1        : 0.8659
```

iv) The fourth representation uses the frequencies of content words rather than all words in the review. Function words are removed using the function-word list specified in the assignment problem, and the remaining content words are represented using their frequencies. Thus, a feature value corresponds to the number of times a particular content word occurs in the review. The vocabulary is constructed from the training data, and words that are unseen during training are excluded.

```
================================================================================
FEATURE 4: Content-word frequency
================================================================================

Fold 1/10
  Vocabulary size: 39,691
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[76, 24], [24, 76]]
Precision : 0.7600
Recall    : 0.7600
Accuracy  : 0.7600
F1        : 0.7600

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[84, 16], [17, 83]]
Precision : 0.8384
Recall    : 0.8300
Accuracy  : 0.8350
F1        : 0.8342

Fold 2/10
  Vocabulary size: 39,851
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[84, 16], [21, 79]]
Precision : 0.8316
Recall    : 0.7900
Accuracy  : 0.8150
F1        : 0.8103

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[89, 11], [21, 79]]
Precision : 0.8778
Recall    : 0.7900
Accuracy  : 0.8400
F1        : 0.8316
```

```
--------------------------------------------------------------------------------
10-FOLD AGGREGATE RESULTS: FEATURE 4
--------------------------------------------------------------------------------

Naive Bayes
------------
Confusion matrix [[TN, FP], [FN, TP]] = [[823, 177], [206, 794]]
Precision : 0.8177
Recall    : 0.7940
Accuracy  : 0.8085
F1        : 0.8057

Logistic Regression
--------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[854, 146], [145, 855]]
Precision : 0.8541
Recall    : 0.8550
Accuracy  : 0.8545
F1        : 0.8546
```

V) Combines content-word filtering with binary Bag-of-Words features. Function words are first removed using the function-word list specified in the assignment, and each remaining content word is represented by a binary value indicating whether it occurs in the review. Thus, the feature value is 1 if the content word occurs at least once and 0 otherwise, regardless of how many times it occurs. Words that are unseen during training are excluded from the feature space.

```
================================================================================
FEATURE 5: Content-word binary
================================================================================

Fold 1/10
  Vocabulary size: 39,691
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[82, 18], [20, 80]]
Precision : 0.8163
Recall    : 0.8000
Accuracy  : 0.8100
F1        : 0.8081

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[85, 15], [13, 87]]
Precision : 0.8529
Recall    : 0.8700
Accuracy  : 0.8600
F1        : 0.8614

Fold 2/10
  Vocabulary size: 39,851
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[86, 14], [21, 79]]
Precision : 0.8495
Recall    : 0.7900
Accuracy  : 0.8250
F1        : 0.8187

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[90, 10], [20, 80]]
Precision : 0.8889
Recall    : 0.8000
Accuracy  : 0.8500
F1        : 0.8421
```

```
--------------------------------------------------------------------------------
10-FOLD AGGREGATE RESULTS: FEATURE 5
--------------------------------------------------------------------------------

Naive Bayes
-----------
Confusion matrix [[TN, FP], [FN, TP]] = [[842, 158], [186, 814]]
Precision : 0.8374
Recall    : 0.8140
Accuracy  : 0.8280
F1        : 0.8256

Logistic Regression
-------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[867, 133], [133, 867]]
Precision : 0.8670
Recall    : 0.8670
Accuracy  : 0.8670
F1        : 0.8670
```

vi) The sixth representation uses word frequencies after applying the negation feature. Negation markers such as `not`, `no`, `never`, and contractions ending in `n't` are detected, and subsequent words are prefixed with `NOT_` until the next punctuation mark. The transformed words are then represented using their frequencies. For example, a phrase such as *“didn't like this movie”* is transformed into *“didn't NOT_like NOT_this NOT_movie”*. The vocabulary is constructed from the training data, and unseen test words are excluded.

```
================================================================================
FEATURE 6: BOW frequency + negation
================================================================================

Fold 1/10
  Vocabulary size: 48,546
  Train=1800 Test=200

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[78, 22], [25, 75]]
Precision : 0.7732
Recall    : 0.7500
Accuracy  : 0.7650
F1        : 0.7614

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[85, 15], [23, 77]]
Precision : 0.8370
Recall    : 0.7700
Accuracy  : 0.8100
F1        : 0.8021

Fold 2/10
  Vocabulary size: 48,743
  Train=1800 Test=200

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[83, 17], [19, 81]]
Precision : 0.8265
Recall    : 0.8100
Accuracy  : 0.8200
F1        : 0.8182

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[89, 11], [17, 83]]
Precision : 0.8830
Recall    : 0.8300
Accuracy  : 0.8600
F1        : 0.8557
```

```
================================================================================
10-FOLD AGGREGATE RESULTS: FEATURE 6
--------------------------------------------------------------------------------

Naive Bayes
------------
Confusion matrix [[TN, FP], [FN, TP]] = [[822, 178], [192, 808]]
Precision : 0.8195
Recall    : 0.8080
Accuracy  : 0.8150
F1        : 0.8137

Logistic Regression
--------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[835, 165], [156, 844]]
Precision : 0.8365
Recall    : 0.8440
Accuracy  : 0.8395
F1        : 0.8402
```

vii) The seventh representation applies the same negation transformation as Feature VI, but represents the resulting words using binary features rather than frequencies. Words occurring within the scope of a detected negation are prefixed with `NOT_` until the next punctuation mark. Each resulting word is then represented by 1 if it occurs in the review and 0 otherwise. The vocabulary is constructed from the training data, and words that are unseen during training are excluded.

```
================================================================================
FEATURE 7: BOW binary + negation
================================================================================

Fold 1/10
  Vocabulary size: 48,546
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[86, 14], [22, 78]]
Precision : 0.8478
Recall    : 0.7800
Accuracy  : 0.8200
F1        : 0.8125

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[82, 18], [18, 82]]
Precision : 0.8200
Recall    : 0.8200
Accuracy  : 0.8200
F1        : 0.8200

Fold 2/10
  Vocabulary size: 48,743
  Train=1800 Test=200

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[89, 11], [20, 80]]
Precision : 0.8791
Recall    : 0.8000
Accuracy  : 0.8450
F1        : 0.8377

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[90, 10], [18, 82]]
Precision : 0.8913
Recall    : 0.8200
Accuracy  : 0.8600
F1        : 0.8542
```

```
--------------------------------------------------------------------------------
10-FOLD AGGREGATE RESULTS: FEATURE 7
--------------------------------------------------------------------------------

Naive Bayes
-----------
Confusion matrix [[TN, FP], [FN, TP]] = [[857, 143], [199, 801]]
Precision : 0.8485
Recall    : 0.8010
Accuracy  : 0.8290
F1        : 0.8241

Logistic Regression
-------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[868, 132], [138, 862]]
Precision : 0.8672
Recall    : 0.8620
Accuracy  : 0.8650
F1        : 0.8646
```

Analysis of the Results obtained so far

```
================================================================================
BEST CONFIGURATIONS
================================================================================
Best Naive Bayes: Feature 5 - Content-word binary (F1=0.8256)
Best Logistic Regression: Feature 5 - Content-word binary (F1=0.8670)
```

Best configuration

```
feature,feature_name,classifier,precision,recall,accuracy,f1
1,BOW frequency (train vocabulary),Naive Bayes,0.8219461697722568,0.794,0.811,0.8077314343845371
1,BOW frequency (train vocabulary),Logistic Regression,0.8371628371628371,0.838,0.8375,0.8375812093953023
2,BOW frequency (train + test vocabulary),Naive Bayes,0.8263598326359832,0.79,0.812,0.8077709611451942
2,BOW frequency (train + test vocabulary),Logistic Regression,0.8371628371628371,0.838,0.8375,0.8375812093953023
3,BOW binary,Naive Bayes,0.8458244111349036,0.79,0.823,0.8169596690796277
3,BOW binary,Logistic Regression,0.8698284561049445,0.862,0.8665,0.8658965344048216
4,Content-word frequency,Naive Bayes,0.8177136972193615,0.794,0.8085,0.8056823947234907
4,Content-word frequency,Logistic Regression,0.8541458541458542,0.855,0.8545,0.8545727136431784
5,Content-word binary,Naive Bayes,0.8374485596707819,0.814,0.828,0.8255578093306287
5,Content-word binary,Logistic Regression,0.867,0.867,0.867,0.867
6,BOW frequency + negation,Naive Bayes,0.8194726166328601,0.808,0.815,0.8136958710976838
6,BOW frequency + negation,Logistic Regression,0.8364717542120912,0.844,0.8395,0.8402190144350422
7,BOW binary + negation,Naive Bayes,0.8485169491525424,0.801,0.829,0.8240740740740741
7,BOW binary + negation,Logistic Regression,0.8672032193158954,0.862,0.865,0.8645937813440322
```

Scores obtained by Naive Bayes and Logistic regression across

Overall, Logistic Regression consistently outperformed Naive Bayes across the feature representations as clearly depicted by the output across features. The best Logistic Regression configuration was Feature V (content word of frequency 1), with an F1 score of 0.8670. The best Naive Bayes configuration was again Feature V, with an F1 score of 0.8256.

This result shows that whether a useful sentiment word appears in a review is more important than how many times it appears. Removing common function words helps the model focus on more meaningful words that are related to the sentiment of the review. Also, using binary features means that a word is counted only once, even if it appears many times. This prevents a frequently repeated word from having too much influence on the prediction. This is helpful for Naive Bayes because repeated words can affect its calculations multiple times. Logistic Regression also benefits because it can learn which of these meaningful words are more strongly associated with positive or negative reviews.

Q2) **Negation Feature**
Negation can change the sentiment of a word. “good” generally expresses positive sentiment, whereas “not good” expresses negative sentiment. To capture this difference, I implemented a negation feature that modifies words occurring within the scope of a negation.

The negation markers considered are:Not, no, never, and the contractions ending in n't, such as didn't, isn't, wasn't, and couldn’t.

When one of these markers is encountered, the subsequent words are prefixed with NOT_ until the next punctuation mark. The negation word itself is not prefixed.
For example:
Original:
didn't like this movie, but I enjoyed it
is transformed after negation processing into:
didn't NOT_like NOT_this NOT_movie, but I enjoyed it.

Thus, the model treats like and NOT_like as different features. This allows it to distinguish between positive expressions such as “like the movie” and their negated forms such as “didn't like the movie”.
Why this feature is useful

Limitation
The implementation uses a simple rule in which negation continues only until the next punctuation mark. Therefore, it may not correctly handle more complex linguistic cases such as double negation, long-distance negation, or complicated sentence structure. Nevertheless, it provides a simple way of incorporating negation information into the Bag-of-Words representation.

Q3)

Feature	Representation	NB Precision	NB Recall	NB Accuracy	NB F1	LR Precision	LR Recall	LR Accuracy	LR F1
Feature 1	BOW frequency (train vocabulary)	0.821946169772257	0.794	0.811	0.807731434384537	0.837162837162837	0.838	0.8375	0.837581209395302
Feature 2	BOW frequency (train + test vocabulary)	0.826359832635983	0.79	0.812	0.807770961145194	0.837162837162837	0.838	0.8375	0.837581209395302
Feature 3	BOW binary	0.845824411134904	0.79	0.823	0.816959669079628	0.869828456104945	0.862	0.8665	0.865896534404822
Feature 4	Content-word frequency	0.817713697219361	0.794	0.8085	0.805682394723491	0.854145854145854	0.855	0.8545	0.854572713643178
Feature 5	Content-word binary	0.837448559670782	0.814	0.828	0.825557809330629	0.867	0.867	0.867	0.867
Feature 6	BOW frequency + negation	0.81947261663286	0.808	0.815	0.813695871097684	0.836471754212091	0.844	0.8395	0.840219014435042
Feature 7	BOW binary + negation	0.848516949152542	0.801	0.829	0.824074074074074	0.867203219315895	0.862	0.865	0.864593781344032

F1, Precision, Recall and Accuracy scores obtained

Feature	Classifier	TN	FP	FN	TP	Confusion Matrix
1	Naive Bayes	828	172	206	794	[[828, 172], [206, 794]]
1	Logistic Regression	837	163	162	838	[[837, 163], [162, 838]]
2	Naive Bayes	834	166	210	790	[[834, 166], [210, 790]]
2	Logistic Regression	837	163	162	838	[[837, 163], [162, 838]]
3	Naive Bayes	856	144	210	790	[[856, 144], [210, 790]]
3	Logistic Regression	871	129	138	862	[[871, 129], [138, 862]]
4	Naive Bayes	823	177	206	794	[[823, 177], [206, 794]]
4	Logistic Regression	854	146	145	855	[[854, 146], [145, 855]]
5	Naive Bayes	842	158	186	814	[[842, 158], [186, 814]]
5	Logistic Regression	867	133	133	867	[[867, 133], [133, 867]]
6	Naive Bayes	822	178	192	808	[[822, 178], [192, 808]]
6	Logistic Regression	835	165	156	844	[[835, 165], [156, 844]]
7	Naive Bayes	857	143	199	801	[[857, 143], [199, 801]]
7	Logistic Regression	868	132	138	862	[[868, 132], [138, 862]]

Confusion matrix obtained for each of the features

Logistic Regression outperforms Naive Bayes for all seven feature representations. The best performance for both classifiers is obtained with Feature V, which combines content-word filtering with binary features. Naive Bayes achieves an F1-score of 0.8256, while Logistic Regression achieves 0.8670. Thus, Logistic Regression improves the best Naive Bayes F1-score by approximately 0.0414.

The results also suggest that binary representations are generally more effective than frequency representations for this dataset. In particular, Feature V performs better than Feature IV for both classifiers, suggesting that the presence of informative content words is more useful than repeatedly counting their occurrences.

Q4) For the error analysis, I compared the best-performing Naive Bayes and Logistic Regression models, both using Feature V: content-word binary features. Feature V was the strongest representation for both models, with an F1-score of 0.8256 for Naive Bayes and 0.8670 for Logistic Regression and hence used to identify the mistakes these models made.

The errors are particularly interesting because a movie review is not necessarily positive just because it contains many positive words. A reviewer may praise an actor, describe a good scene, or acknowledge an interesting idea while still giving the movie an overall negative evaluation. This is one of the main limitations of a simple bag-of-words representation.

1. cv003_12683.txt

True label: Negative
Naive Bayes: Positive
Logistic Regression: Negative

The review of Quest for Camelot contains words and descriptions that can look positive in isolation: it talks about a “promising” production, a “lively cast”, a “colorful” palette, and other aspects of animation and the characters. However, the overall judgement of the reviewer is clearly negative: the movie is described as dull and not a serious competitor to better animated films.

This illustrates an important problem with Naive Bayes. Naive Bayes considers the evidence provided by individual words through their class-conditional probabilities. If enough words occurring in the review are more frequently associated with positive reviews, their combined contribution can push the prediction toward positive.
Logistic Regression handles this somewhat differently. It learns a weight for each feature and makes its decision from the combined weighted evidence. Therefore, highly diagnostic negative content words can outweigh less informative positive-looking words.
The important linguistic point is that mentioning something positively is not necessarily the same as expressing a positive overall opinion. A reviewer can say that an actor is good while simultaneously saying that the film itself is bad.

2. cv011_13044.txt

True label: Negative
Naive Bayes: Negative
Logistic Regression: Positive

The review contains several potentially positive expressions, including discussion of a “good idea” and positive aspects of the production. However, these comments are embedded inside an overwhelmingly negative evaluation of the movie. The reviewer describes the film as predictable, lacking surprises and laughs, and repeatedly criticizes its execution.
This is a classic example of mixed sentiment. The reviewer is not saying:
“Everything about this movie is bad.”
Instead, the reviewer acknowledges some good aspects but concludes that the movie as a whole is unsuccessful.
Here Naive Bayes happens to classify the review correctly. This shows why it would be incorrect to claim that Logistic Regression is always superior. Logistic Regression performed better overall, but individual reviews can still favour Naive Bayes depending on which features are present.
The interesting linguistic phenomenon is the distinction between aspect-level sentiment and document-level sentiment. A sentence can praise one aspect of a movie without changing the overall polarity of the review.

3. cv018_21672.txt

True label: Negative
Naive Bayes: Negative
Logistic Regression: Positive

The review contains some positive or mildly positive language about the movie and its genre, which can provide evidence for a positive classification. However, the reviewer ultimately gives a negative judgement. In particular, the review asks whether the journey to the ending is worth experiencing and answers negatively: “no, it definitely is not.”
This example demonstrates the difficulty of handling negation and discourse-level sentiment using a simple bag-of-words representation. The important sentiment is not determined by the individual word worth, but by the larger construction “not worth it.” A binary content-word representation does not explicitly represent the relationship between not and worth.

Also notice that sentiment can change as the review progresses. Earlier positive or neutral descriptions may suggest one polarity, while a later explicit evaluation reverses the overall interpretation. Logistic Regression therefore incorrectly predicts positive for this review, whereas Naive Bayes correctly predicts the negative label.

The models apparently struggle with the following linguistic cases:
negation: “not good”
contrast: “good acting, but a terrible movie”
mixed sentiment: praising one aspect while criticizing another
discourse: distinguishing a description from the reviewer's final judgement
context: understanding which words are actually important to the overall opinion

Conclusion: The results suggest that Logistic Regression is the better classifier for the given representation, but the representation itself remains the major linguistic limitation.

Feature V works particularly well because removing function words and using binary content-word features focuses the model on informative lexical items, but it still cannot fully capture the relationships between those words.

In short, the errors show an important distinction between “what words occur in a review” and “what the reviewer actually means.” That distinction explains both the remaining errors and why a more linguistically informed representation could potentially improve performance further.

Bonus Question (Dataset used: Opinion lexicon of Hu and Liu (2004))

Assumptions and Preprocessing

1)Tokenization: The same tokenization procedure used for the main experiments is applied to the reviews for the lexicon-based features. Words are treated as lowercase alphabetic tokens, with contractions such as didn't retained as a single token where applicable.

2)Case normalization: The review text and the Hu & Liu lexicon entries are converted to lowercase before matching. Thus, lexicon matching is case-insensitive.
-Lexicon matching: A word contributes to the positive count if it exactly matches an entry in the Hu & Liu positive-word list, and similarly for the negative count. No stemming or lemmatization is applied.

3)Punctuation: Punctuation marks are not counted as lexicon entries. Only word tokens are considered when computing the positive and negative lexicon counts.
Feature construction: Two additional numerical features are used:
POS_LEXICON_COUNT: number of positive Hu & Liu words occurring in the review.
NEG_LEXICON_COUNT: number of negative Hu & Liu words occurring in the review.
These features are added to the existing Feature V representation rather than replacing the original content-word binary features.

4)Multiple occurrences (Assumption): If a lexicon word occurs multiple times in a review, each occurrence contributes to its corresponding lexicon count.

5)External lexicon: The Hu & Liu (2004) positive and negative word lists are treated as fixed external resources. They are not learned or modified using the movie-review dataset.

6)Evaluation: The same predefined 10-fold evaluation procedure as the main experiments is used. In each fold, 1800 reviews are used for training and 200 reviews for testing, and the process is repeated for all ten folds.

7)Character encoding: The movie-review files are read using the encoding supported by the original dataset files, and the lexicon files are read as text resources. This avoids treating non-ASCII characters as numerical or lexical features incorrectly.
Negation: The lexicon counts are based on direct Hu & Liu word matches. The lexicon itself does not inherently determine the scope of negation; therefore, a word such as good is counted as positive when it directly matches the positive lexicon, even if it occurs in a more complex negated construction. The separate negation feature experiments in the main assignment address negation explicitly.
We use the similar 10-fold train and test dataset used for the previous problems.

OUTPUTS OBTAINED

```
Positive lexicon entries: 2006
Negative lexicon entries: 4783
Added features: POS_LEXICON_COUNT, NEG_LEXICON_COUNT

Fold 1/10

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[83, 17], [18, 82]]
Precision : 0.8283
Recall    : 0.8200
Accuracy  : 0.8250
F1        : 0.8241

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[90, 10], [22, 78]]
Precision : 0.8864
Recall    : 0.7800
Accuracy  : 0.8400
F1        : 0.8298

Fold 2/10

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[87, 13], [18, 82]]
Precision : 0.8632
Recall    : 0.8200
Accuracy  : 0.8450
F1        : 0.8410

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[95, 5], [16, 84]]
Precision : 0.9438
Recall    : 0.8400
Accuracy  : 0.8950
F1        : 0.8889

Fold 3/10
```

```
Confusion matrix [[TN, FP], [FN, TP]] = [[87, 13], [16, 84]]
Precision : 0.8660
Recall    : 0.8400
Accuracy  : 0.8550
F1        : 0.8528

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[79, 21], [10, 90]]
Precision : 0.8108
Recall    : 0.9000
Accuracy  : 0.8450
F1        : 0.8531

Fold 4/10

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[79, 21], [18, 82]]
Precision : 0.7961
Recall    : 0.8200
Accuracy  : 0.8050
F1        : 0.8079

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[83, 17], [12, 88]]
Precision : 0.8381
Recall    : 0.8800
Accuracy  : 0.8550
F1        : 0.8585

Fold 5/10
```

```
Fold 8/10

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[89, 11], [18, 82]]
Precision : 0.8817
Recall    : 0.8200
Accuracy  : 0.8550
F1        : 0.8497

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[76, 24], [5, 95]]
Precision : 0.7983
Recall    : 0.9500
Accuracy  : 0.8550
F1        : 0.8676

Fold 9/10

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[85, 15], [24, 76]]
Precision : 0.8352
Recall    : 0.7600
Accuracy  : 0.8050
F1        : 0.7958

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[88, 12], [28, 72]]
Precision : 0.8571
Recall    : 0.7200
Accuracy  : 0.8000
F1        : 0.7826

Fold 10/10

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[92, 8], [16, 84]]
Precision : 0.9130
Recall    : 0.8400
Accuracy  : 0.8800
F1        : 0.8750
```

```
  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[72, 28], [7, 93]]
Precision : 0.7686
Recall    : 0.9300
Accuracy  : 0.8250
F1        : 0.8416

Fold 6/10

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[83, 17], [18, 82]]
Precision : 0.8283
Recall    : 0.8200
Accuracy  : 0.8250
F1        : 0.8241

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[67, 33], [4, 96]]
Precision : 0.7442
Recall    : 0.9600
Accuracy  : 0.8150
F1        : 0.8384

Fold 7/10

  Naive Bayes
--------------
Confusion matrix [[TN, FP], [FN, TP]] = [[85, 15], [18, 82]]
Precision : 0.8454
Recall    : 0.8200
Accuracy  : 0.8350
F1        : 0.8325

  Logistic Regression
----------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[95, 5], [26, 74]]
Precision : 0.9367
Recall    : 0.7400
Accuracy  : 0.8450
F1        : 0.8268

Fold 8/10
```

```
Fold 10/10

  Naive Bayes
-------------
Confusion matrix [[TN, FP], [FN, TP]] = [[92, 8], [16, 84]]
Precision : 0.9130
Recall    : 0.8400
Accuracy  : 0.8800
F1        : 0.8750

  Logistic Regression
---------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[82, 18], [6, 94]]
Precision : 0.8393
Recall    : 0.9400
Accuracy  : 0.8800
F1        : 0.8868

--------------------------------------------------------------------------------
10-FOLD POOLED RESULTS: FEATURE V + HU & LIU
--------------------------------------------------------------------------------

Naive Bayes
-----------
Confusion matrix [[TN, FP], [FN, TP]] = [[854, 146], [180, 820]]
Precision : 0.8489
Recall    : 0.8200
Accuracy  : 0.8370
F1        : 0.8342

Logistic Regression
-------------------
Confusion matrix [[TN, FP], [FN, TP]] = [[827, 173], [136, 864]]
Precision : 0.8332
Recall    : 0.8640
Accuracy  : 0.8455
F1        : 0.8483

10-fold macro-average:
  NB: Precision=0.8497, Recall=0.8200, Accuracy=0.8370, F1=0.8343
  LR: Precision=0.8423, Recall=0.8640, Accuracy=0.8455, F1=0.8474
```

Classifier	Precision	Recall	Accuracy	F1
Naive Bayes + Hu & Liu	0.8489	0.8200	0.8370	0.8342
Logistic Regression + Hu & Liu	0.8332	0.8640	0.8455	0.8483

Final Outputs Obtained